



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۲ دی

فصل ششم: *link layer* اسلاید ۲۵ تا ۵۱ [۱]

slotted ALOHA

این روش که یک روش *random access* است به این شکل عمل میکند. اندازه فریم‌های داده مساوی است و زمان را به تعدادی اسلات مساوی تقسیم میکنیم (زمان اسلات زمان لازم برای ارسال یک فریم است). هر کسی میتواند دیتای خود را فقط در شروع اسلات‌ها بفرستد (در صورت کامل حاضر و آماده بودن دیتا) اگر *collision* رخ نداد، دیتاهای بعدی در اسلات‌های بعدی فرستاده میشود و اگر *collision* رخ داد، متوجه میشود و همان دیتا را باید باز ارسال کند. دوباره همان دیتا را با احتمال p در اسلات بعدی باز ارسال میکند. پس اینگونه بعضی اسلات‌ها *collision* رخ میدهد و بعضی اسلات‌ها *empty* است و بعضی هم *success*. بهره‌وری را تعریف میکنیم که در طولانی مدت چه تعدادی از اسلات‌ها *success* میباشند. خوبیش اینه که وقتی یکی میفرسته از کل پهنای باند میتونه استفاده کنه. البته وقتی که یکی همان اول فهمید *collision* رخ داده لازم نیست کل دیتا را ارسال کند و سریع ارسال ادامه دیتا را قطع میکند. به صورت کاملاً *decentralized* است اما فقط باید تایم شروع اسلات‌ها را با هم هماهنگ کنند. الگوریتم ساده‌ای است.

بهره‌وری را برای شرایط *saturation* که تعداد زیادی گره داریم و هر کدام تعداد زیادی دارند. احتمال ارسال موفق یک گره $p(1-p)^{N-1}$ است یعنی اون گره با p بفرستد و بقیه با احتمال $1-p$ نفرستند. حالا این برای هر کدام از N گره رخ دهد، *success* میشود. پس در کل به احتمال $Np(1-p)^{N-1}$ به *success* میرسیم. اگه مشتق بگیریم تا بهترین p بدست بیاد که این احتمال را بیشینه کند، که مقدار آن $1/N$ خواهد بود. با فرض اینکه N را به بی نهایت میل دهیم، p برابر با $1/e \approx 0.37$ خواهد بود.

unslotted(pure) ALOHA

اینجا دیگر لازم نیست که نود ها سر تایم اسلات ها با هم هماهنگ شوند و هر کسی هر موقعی خواست میتواند بفرستد. اما اینجا بازه آسیب پذیری دو برابر اندازه فریم میباشد (در روش قبلی، اندازه اسلات که همان اندازه فریم میشد، میشد بازه *vulnerability* ما اما اینجا دو برابر است) یعنی احتمال برخورد افزایش میابد، یعنی اگر یکی در t_0 فریم بفرستد، اگر یکی در بازه $[t_0 - 1, t_0 + 1]$ ارسال کند به *collision* میخوریم.

احتمال موفقیت هر نود اینطوری حساب میشه که هر نود با احتمال p بفرستد، و تمام نود های دیگر با احتمال $1 - p$ در بازه $t_0 - 1, t_0$ بفرستند و باز تمام نود های دیگر با احتمال $1 - p$ در بازه $t_0, t_0 + 1$ بفرستند. پس در کل احتمال موفقیت یک نود، $p(1 - p)^{N-1}(1 - p)^{N-1} = p(1 - p)^{2N-1}$ است. اگر این را برای همه N نود در نظر بگیریم (ضرب در N) و نسبت به p مشتق بگیریم و N را به بینهایت میل دهیم به مقدار $1/2e \approx 0.18$ برای p بهینه میرسیم. که بدتر از *slotted ALOHA* شد و نصف اون شد. چون بازه آسیب پذیری دو برابر اون بود.

CSMA = carrier sense multiple access

این هم یک نوع دیگری از الگوریتم های *random access* است. هر نود قبل ارسال اول به کانال گوش میدهد و *sense* میکند. اگر بیکار باشد فریم را میفرستد ولی اگر *busy* باشد، صبر میکند.

هنور شانس برخورد هست به خاطر *propagation delay*. تاخیر انتشار یعنی دو گره هنوز ارسال هم را متوجه نشده است.

وقتی هم *collision* رخ میدهد و بعد از مدتی به گوش تمام نود ها میرسد. در *CSMA* با اینکه نود ها میفهمند که برخورد رخ داده ولی باز به ارسال میدهند که عمل بیهوده ای است. پس فاصله بین گره و تاخیر انتشار نقش اصلی در بهره وری پروتکل *CSMA* دارد.

CSMA/CD(collision detection)

چون *collision* زود تشخیص داده میشود، به محض آنکه تشخیص دادن ارسال ادامه داده را متوقف میکنند. در محیط های *LAN* با سیم به راحتی میتوان آن را پیاده سازی کرد. همزمان با ارسال، گوش میدهد و قدرت سیگنال ارسالی و دریافتی را با هم مقایسه میکند. اگر به هم ریخته بود، ارسال را متوقف میکند.

در *LAN* های بی سیم، این تشخیص سخت است. چون قدرت سیگنال های ارسالی بیشتر از سیگنال دریافتی است و متوجه تخریب سیگنال دریافتی شاید نشویم.(?)

این الگوریتم در اترنت اینگونه است که اول دیتاگرام لایه بالایی را کارت شبکه میگیرد و به فریم تبدیل میکند. اگر لینک *idle* بود میفرستد ولی اگر نبود صبر میکند. وقتی فرستاد اگر رسید که هیچی اما اگر برخورد داشت، ارسال داده را متوقف میکند و یک سیگنال *jam* میفرستد که خیلی قوی است. نکته این سیگنال این است که شاید برخی نود ها، سیگنال تخریب شده و *collision* شده را متوجه نشوند و این سیگنال خیلی قوی است و به هم اطلاع میدهد *collision* رخ داده. حال فرض کنید که m بار پیاپی برخورد رخ داده. برای بار بعدی، کارت شبکه یک عدد از بین 0 تا $2^m - 1$ انتخاب میکند مثلاً عدد k و به اندازه $k \times \text{bit times}$ صبر میکند. بعد از آن دوباره به آن مرحله *sense* کردن لینک و چک کردن اینکه آیا لینک *idle* است یا نه که بتواند بفرستد، میرود. اون bit times بستگی به نرخ خط و سرعت کارت شبکه دارد و مثلاً برای *Ethernet* ۱۰ مگابیت در ثانیه، برابر 0.1 میگرثانیه است.

کارآمد پروتکل CSMA/CD

کارآمدی این پروتکل نسبت معکوس دارد با نسبت $\frac{t_{prop}}{t_{trans}}$ و فرمول آن به این شکل است:

$$efficiency = \frac{1}{1 + 5 \frac{t_{prop}}{t_{trans}}}$$

ما میخواهیم که کارآمدی به ۱ برسد. برای این باید یا t_{prop} صفر شود یعنی تاخیر انتشار بین دو نود همسایه صفر شود و یا یعنی آنها باید خیلی بهم نزدیک باشند. و یا اینکه t_{trans} به بینهایت میل کند. یعنی اندازه فریم خیلی بزرگ باشد.

به توجه به این فرمول، *performance* بهتری از *ALOHA* دارد. هم ارزان هم ساده و هم توزیع شده است. پروتکل *CSMA/CA* در 802.11 به کار میرود.

پروتکل های taking turn

همانطور که دیدید پروتکل های *channel partitioning* با اینکه در لود بالا به صورت خوب و *fair* و کارآمد عمل میکنند اما در لود پایین، کار آمد نیستند. حتی در *TDMA* باید منتظر بمانی تا اسلات شود. در پروتکل های *random access* در لود پایین خوب عمل میکنند اما در لود بالا، سربار زمانی *collision* خیلی زیاد خواهد بود.

پروتکل *taking turn* سعی میکنه ویژگی های مثبت هر دو روش را انجام دهد

پروتکل *polling*

یک نود مستر داریم و هر چند وقت یک بار، از نود های *slave* میپرسد که آیا داده ای برای ارسال دارند یا خیر. برای زمانی که نود های *slave* ضعیف هستند، به کار میرود. نکته های منفی، اینکه نود مستر، *single point of failure* ایجاد میکند و همینطور سر بار زیادی خود *polling* مخصوصا زمانی که تعداد *slave* ها زیاد شود و داده ای برای ارسال نداشته باشند. پس *scalable* نیست. مسئله بعد، *latency* است چون مدتی طول میکشد به نوبت یک نود برسد مخصوصا وقتی گره های قبلی دیتا برای ارسال دارند.

پروتکل *token passing*

اینجا یک توکن نوبتی به صورت دوری به گره ها داده میشود. اگر داده نداشته باشند به نود بعدی میدهند و اگر داده داشته باشند، توکن را نگه میدارند و داده را ارسال میکنند به مقصد تا زمانی که همان داده دوباره بدست خودشان برسد و آنوقت توکن را آزاد میکنند. اینجا باز هم مشکلاتی دارد. مثلا چون یک توکن داریم، به نوعی توکن ما *single point of failure* است. همچنین اینجا این توکن میتواند برای ما سر بار ایجاد کند و همچنین هنوز هم مشکل *latency* را مثل قبل داریم.

درباره LAN

آدرس *MAC* یا *LAN* یا *EHTERNET* به نوعی برای آدرس دهی درون یک شبکه *LAN* و برای انتقال از یک *interface* به یک *interface* فیزیکی مجاور استفاده میشود. ۴۸ بیت است برعکس آدرس *IP* که ۳۲ بیت است. آدرس *MAC* به صورت ۱۲ تا عدد هگزادسیمال است که با خط تیره از هم جدا شدند. در *IP* با نقطه از هم جدا میشدند.

آدرس های مک توسط *IEEE* ارائه میشود.

به صورت *hardcoded* یا *burned* روی اون *NIC* است. البته بعضی وقتا به صورت نرم افزاری قابل تنظیم میباشد. احتمالا به خاطر همین *hardcoded* بودن، آدرس مک *portability* بالایی دارد یعنی مثلا کارت شبکه را میتوانی به یک *LAN* و هاست دیگر متصل کنی و هنوز همان مک را دارد. ولی آدرس *IP* اینگونه نیست و در محیط ها و مکان های مختلف، متفاوت میتواند باشد.

پروتکل *ARP = address resolution protocol*

این پروتکل از روی آدرس *IP* آدرس مک آن دستگاه را بدست میآورد.

درواقع یک *ARP table* داریم که در هر نود (هاست و روتر) وجود دارد و در آن نگاشت بین آدرس *IP* و آدرس مک و یک *TTL* وجود دارد (یجور تاریخ انقضای آن که به صورت خودکار کم میشود) که به این *soft state* میگویند یعنی اطلاعات پس از مدتی پاک میشود مگر اینکه رفرش شوند.

ARP یک پروتکل از نوع *plug and play* است یعنی بدون نیاز به نود خارجی، و دخالت سرپرست شبکه، جدول خود را میسازد.

ارسال داخل LAN

حالا فرض میکنید *A* میخواهد پیامی را به *B* بفرستد که درون *LAN* است. ولی آدرس مک آن را ندارد. پس یک پیام *ARP query* را به همه برودکست میکند و آدرس مک را همه را ۱۲ تا *F* میگذارد. $FF - FF - FF$ (احتمالا همان آدرس *IP* نود *B* را میگذارد و به جای آدرس مکش این را میگذارد) وقتی *B* پیام را گرفت، یک پیام جواب به *unicast A* میکند همچنین آدرس مک *A* را در جدول آرپ خود ذخیره میکند. *A* با دریافت پاسخ، آدرس مک او را ذخیره میکند.

ارسال به LAN دیگر

فرض کنید که از *A* به یک نود *B* خارج از این *LAN* که یک روتر بینشان هست، میخواهیم داده ای ارسال کنیم. یعنی ارسال داده *A* به *B* از طریق *R* انجام میشود.

نکته روتر این است که *broadcast domain* را میشکند و آن را محدود به یک *LAN* میکند تا پیام همه پخشی در طول شبکه منتشر نشود.

نود *A* آدرس *IP* نود *B* را میداند.

نود *A* آدرس *IP* روتر *first - hop* را از روی *dhcp* یاد گرفته است. و همچنین با کمک پروتکل *ARP* آدرس مک *R* را میداند.

حالا وقتی که *A* پیامی میخواهد بفرستد، آدرس ایپی مبدا و مقصد و مک مبدا را میگذارد ولی برای مک مقصد، آدرس مک روتر در سمت *interface* اون *LAN* خودش را میگذارد.

روتر هم چون آدرس مک مقصد با مک او یکیست، فریم را مطالعه میکند و یک فریم جدید میسازد که آدرس مک مبدا، آدرس مک *interface* روتر در سمت *LAN* نود *B* و آدرس مک مقصد هم آدرس مک *B* است.

مراجع